Machine Learning Lectures

Classification part 2

Classification – Part 2

By:
Dr. Andrey Holz
https://www.linkedin.com/in/andrey-holz

Machine Learning Lectures

Classification – Part 2

By:
Dr. Andrey Holz
https://www.linkedin.com/in/andrey-holz

Common Challenges for Classifica ks

Previously on Classification 1:

1. **Class Imbalance**:
    1. When classes are unevenly represented, leading to bias toward the majority class.
    2. **Solution**: Resampling techniques, SMOTE, or weighted loss functions.
2. **Multiclass & Multilabel**:
    1. **Multiclass**: More than two categories (e.g., cat, dog, bird).
    2. **Multilabel**: Multiple labels per instance (e.g., "indoor" and "nature").
    3. **Challenge**: Requires complex algorithms for overlapping labels.
3. **Ordinal Classes**:
    1. Ordered categories (e.g., low, medium, high) where distances aren't equal.
    2. **Solution**: Specialized models for ordinal data.
4. **High-Dimensional Data**:
    1. Many features increase computational cost and risk overfitting.
    2. **Solution**: Dimensionality reduction (e.g., PCA).
5. **Interpretability**:
    1. Complex models (e.g., neural networks) lack transparency.
    2. **Solution**: Interpretable models or explanation tools like SHAP, LIME.

Agenda

1. Binary Imbalanced Classification
    1. Introduction to Binary Imbalance
    2. Evaluation Metrics for Imbalanced Binary Classification
    3. Techniques to Address Imbalance
2. Introduction to Multiclass and Multilabel Classification
    1. Evaluation Metrics for Multiclass Classification
    2. Evaluation Metrics for Multilabel Classification
    3. Brief Introduction to Ranking Problems and Metrics
3. Key Notes

Binary Imbalanced Classification

Introduction to Binary Imbalance

Definition:

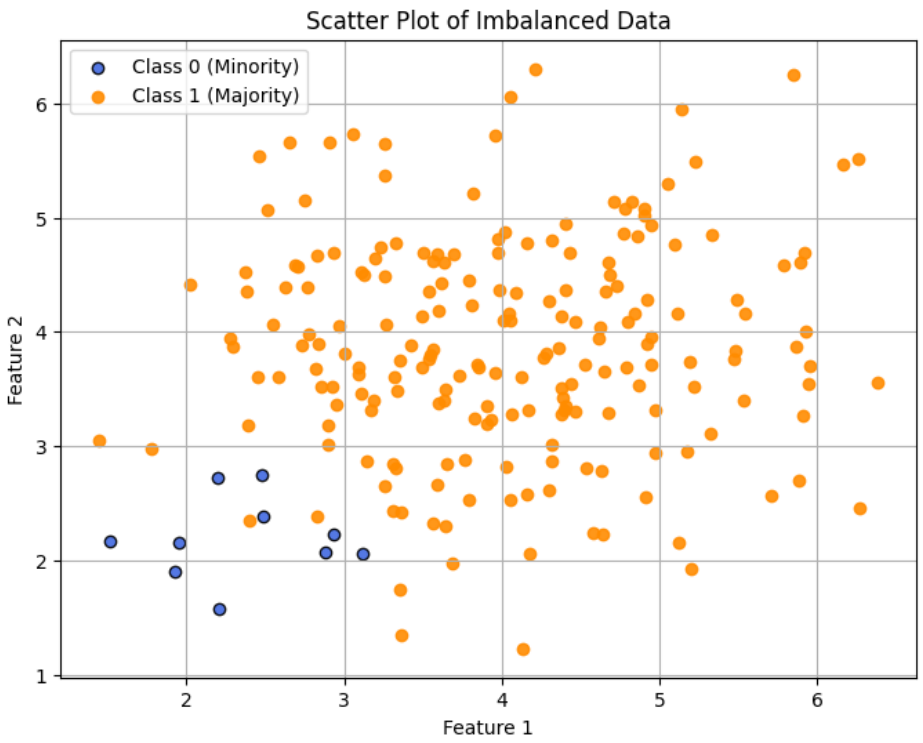
- Imbalance occurs when one class significantly outnumbers the other.

Examples:

- Fraud detection, where fraudulent transactions are rare.
- Medical diagnosis for rare diseases.

Impact on Machine Learning Models:

- Bias towards the majority class.
- Poor predictive performance on the minority class.



Evaluation Metrics for Imbalanced Binary Classification

$$Accuracy = \frac{TP + \boldsymbol{TN}}{TP + FP + FN + \boldsymbol{TN}}$$

High accuracy can be achieved by predicting ONLY the majority class.

Better Metrics:

- **Precision**: $Precision = TP/(TP + FP)$

  Useful in cases where the cost of false positives is high

- **Recall (Sensitivity)**: $Recall = TP/(TP + FN)$

  Important when missing a positive case has a high cost

- **F1-Score**: Harmonic mean of precision and recall.
- **ROC Curve and AUC**
- **PR Curve (Precision-Recall Curve)**

Why ROC AUC is Good for Imbalanced Data

$$ROC\ AUC = P(score(X_{pos}) > score(X_{neg}))$$

The ROC AUC is equivalent to **the probability that a randomly chosen positive instance is ranked higher than a randomly chosen negative instance.**

The ROC curve is generated by plotting the True Positive Rate (TPR) against the False Positive Rate (FPR) at various threshold settings:

$$TPR = TP/(TP + FN) \qquad FPR = FP/(FP + TN)$$

Considerations:

Threshold Independence:

- Evaluates model performance across all classification thresholds.

Class Distribution Independence:

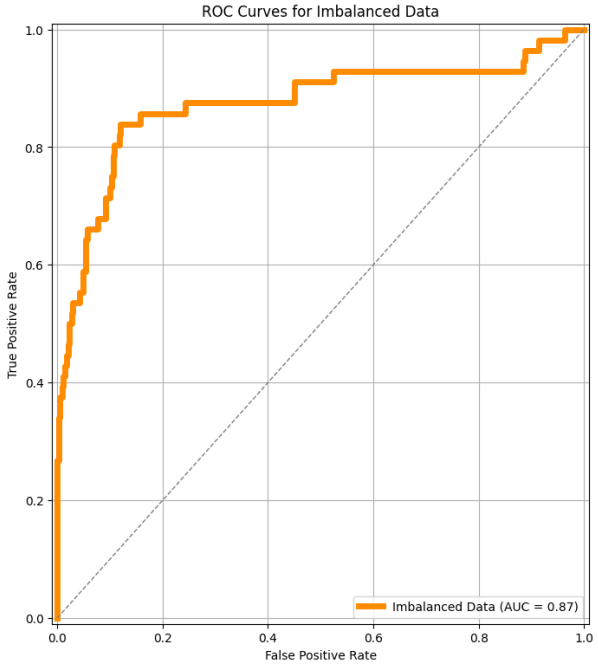
- Measures the ability to distinguish between classes regardless of imbalance.

Focus on Ranking Ability:

- Reflects the probability that a positive instance ranks higher than a negative one.

Limitations with Extreme Imbalance:

- In cases of extreme imbalance, ROC AUC might be overly optimistic because the FPR can be deceptively low due to the large number of true negatives.



Why Precision-Recall Curve is Good for Imbalanced Data

The precision-recall curve shows the tradeoff between precision and recall for different thresholds.

$$\boldsymbol{Precision} = TP/(TP + FP)$$

$$\boldsymbol{Recall} = TP/(TP + FN)$$

Average precision (AP) summarizes such a plot as the weighted mean of precisions achieved at each threshold, with the increase in recall from the previous threshold used as the weight

$$\boldsymbol{AP} = \sum_n (R_n - R_{n-1})P_n$$

Considerations:

Focus on Minority Class Performance

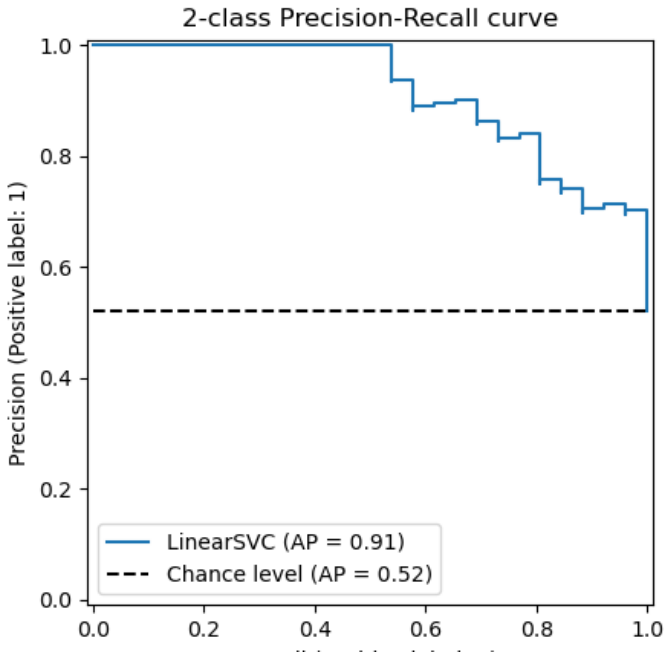
- Directly evaluates the trade-off between Precision and Recall for the positive class.

Sensitive to Class Imbalance

- More informative than ROC AUC for imbalanced datasets.

Highlights Model's Ability to Identify Minority Class

- Emphasizes the performance on detecting positive instances.



Techniques to Address Imbalance

Data-Level Methods

Resampling Techniques:

- Oversampling the minority class (e.g., SMOTE).
- Undersampling the majority class.

Algorithm-Level Methods

Cost-Sensitive Learning:

Assign higher misclassification costs to the minority class.

Oversampling & Undersampling

OVERSAMPLING	UNDERSAMPLING
Generates additional samples for the minority class to balance the dataset. **Techniques:** • **Random:** randomly duplicates instances from the minority class. • **Synthetic Minority Oversampling Technique (SMOTE)**:Generates synthetic samples by interpolating between existing minority samples. **Advantages:** Improves model's focus on the minority class. **Disadvantages:** Can lead to overfitting, especially with random oversampling.	**Reduces the number of majority class instances to balance the dataset.** **Techniques:** • **Random:** Randomly removes instances from the majority class. • **Cluster-Based:** Clusters majority samples and selectively reduces them while retaining representative points. **Advantages:** Reduces bias towards the majority class. **Disadvantages:** Loss of information from the majority class, which can lead to underfitting.

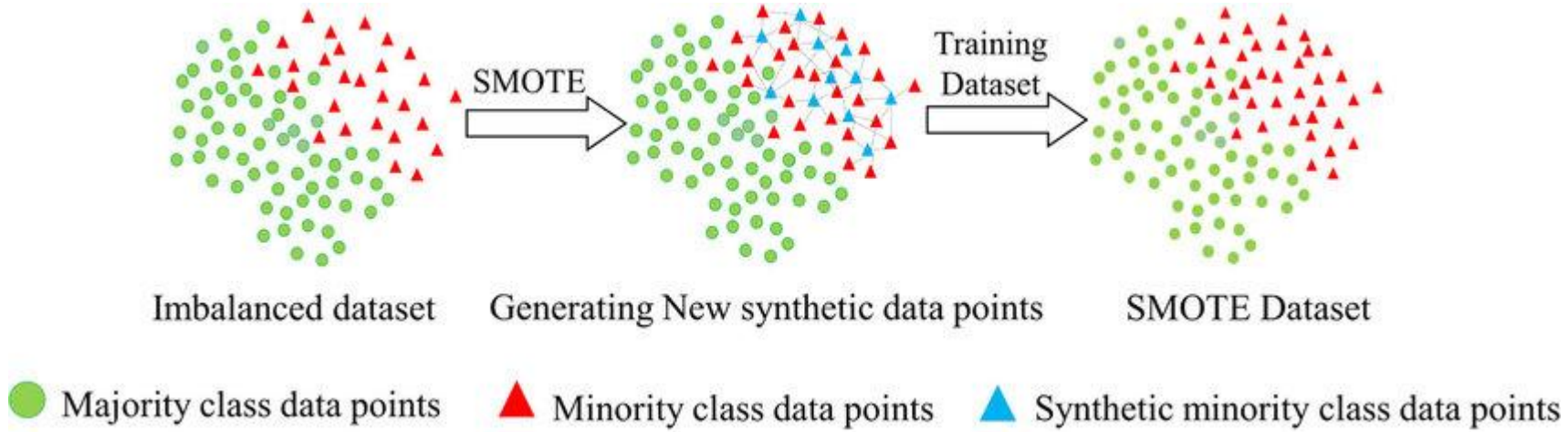
Synthetic Minority Oversampling Technique (SMOTE)

Selects pairs of similar minority class instances.

Generates synthetic samples by randomly interpolating between them.

Mathematical Foundation:

- Given two points x_i and x_j :New sample $x_{new} = x_i + \lambda(x_j - x_j)$, where λ is a random number between 0 and 1.



https://pub.aimind.so/synthetic-minority-over-sampling-technique-smote-empowering-ai-through-imbalanced-data-handling-d86f4de32ea3

Choosing Between Oversampling and Undersampling

When to Use Oversampling

- Dataset is small, and minority class needs more representation.
- Model benefits from seeing more minority examples.

When to Use Undersampling

- Dataset is large, and removing some majority samples won’t impact generalization.
- Want to avoid the risk of overfitting due to duplicating minority samples.

Cost-Sensitive Learning

A technique that adjusts the model to account for the varying costs of different types of errors, especially useful for imbalanced datasets.

Goal:

Minimize overall cost by assigning higher penalties to misclassifications of the minority class (or costly errors).

How It Works:

- **Class Weights:** Increase weight for the minority class, encouraging the model to focus on it.

- **Cost Matrix:** Define costs for false positives (FP) and false negatives (FN)
  - **Example:** $C_{FN} > C_{FP}$ in fraud detection where missing fraud (FN) is costlier than a false alarm (FP).

Applications: Fraud Detection, Medical Diagnosis, Loan Approval

Example Formula, Weighted Loss: $L = \sum_{i=1}^{N} C(y_i, \hat{y}_i) \text{ Loss } (y_i, \hat{y}_i)$

Here, $C(y_i, \hat{y}_i)$ represents the misclassification cost based on the true and predicted classes.

Models for Imbalance Classification

Model	Challenges	Solutions
K-Nearest Neighbors (KNN)	Bias towards majority class	**Weighted KNN**: Assign higher weights to minority class neighbors **Distance Metrics**: Experiment with different metrics (e.g., Euclidean, Manhattan)
Decision Trees	Overfitting to majority class	**Class Weighting**: Adjust the cost of misclassifying minority class instances. **Pruning**: Reduce the complexity of the tree to avoid overfitting.
Logistic Regression	Bias towards majority class	**Class Weighting**: Adjust the loss function to penalize misclassification of minority class instances more heavily. **Threshold Tuning:** Lower the classification threshold to increase sensitivity to the minority class.
Naive Bayes	Sensitivity to rare events in minority class	**Adjust Prior Probabilities**: Assign higher probabilities to the minority class to account for its underrepresentation.
Support Vector Machines (SVM)	Bias towards majority class	**Class Weighting**: Assign higher weights to minority class instances. **One-Class SVM**: Treat the minority class as anomalies and train a model to detect them.

Live Coding & Demos

Introduction to Multiclass & Multilabel Classification

Definition and Challenges

Multiclass Classification

- Problems with more than two classes.
- Each instance belongs to one class.

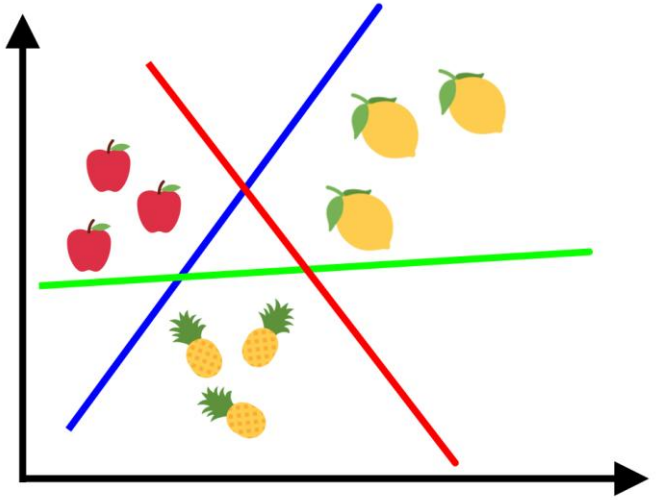
Multilabel Classification

- Instances can belong to multiple classes simultaneously.

Challenges

- Increased complexity.
- Imbalance across multiple classes.

Understanding Multiclass Classification



Definition:

Classification problem where each instance belongs to one of three or more classes.

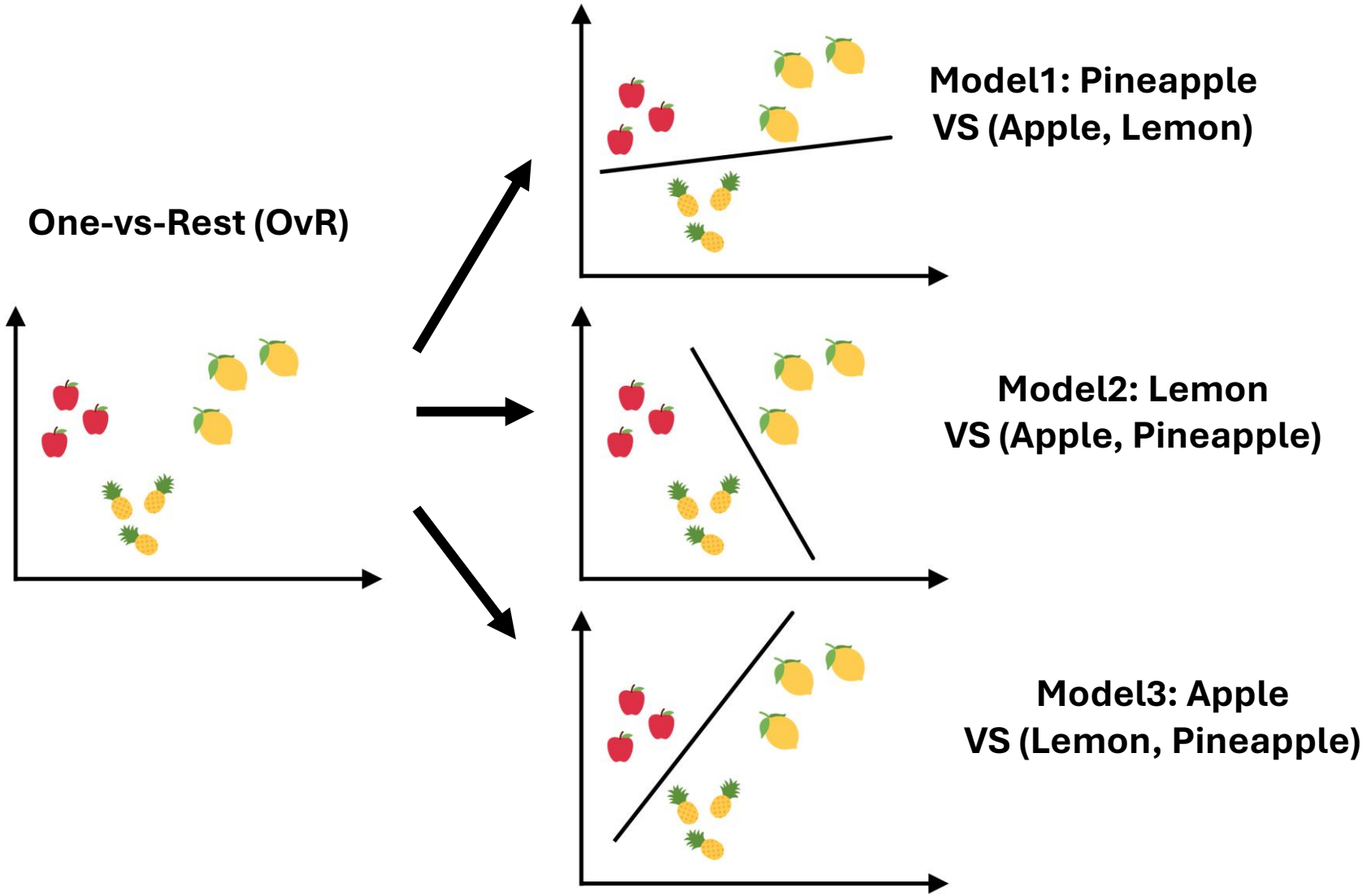
Approaches:

One-vs-Rest (OvR)	One-vs-One (OvO)	Multinomial Models
Multiple binary classification problems. **K Classifiers**	Constructs classifiers for every pair of classes. **K(K-1)/2 Classifiers**	Extends binary classification algorithms to handle multiple classes directly.

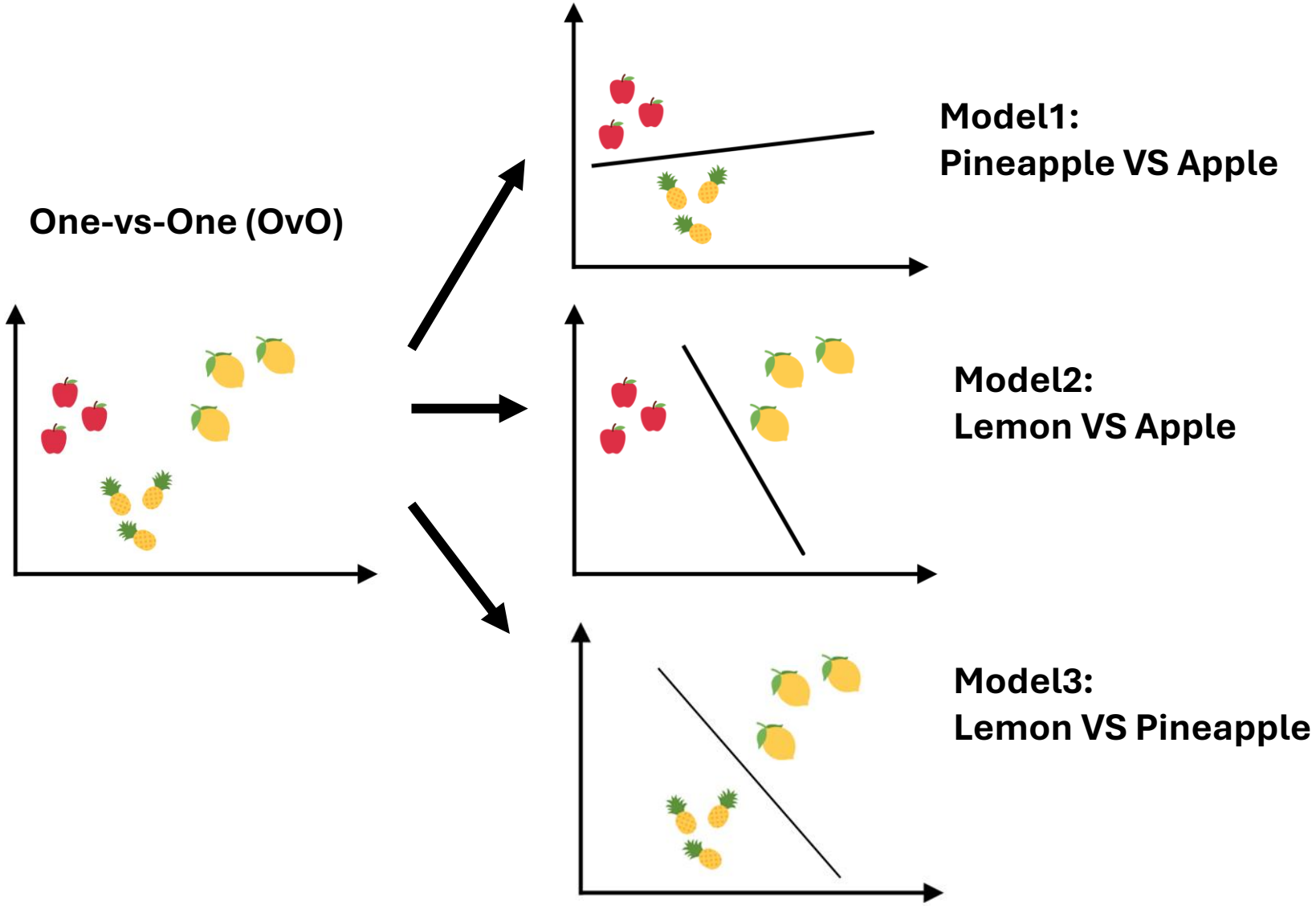
Example:

Classifying handwritten digits (0-9) using Multinomial Logistic Regression.

One-vs-Rest


One-vs-Rest (OvR)
Model1: Pineapple
VS (Apple, Lemon)
Model2: Lemon
VS (Apple, Pineapple)
Model3: Apple
VS (Lemon, Pineapple)

One-vs-One


One-vs-One (OvO)
Model1:
Pineapple VS Apple
Model2:
Lemon VS Apple
Model3:
Lemon VS Pineapple

Multiclass Classification Evaluation Metrics

Confusion Matrix Extension

Rows and columns correspond to each class.

Accuracy

Overall correctness: $\frac{Correct\ predictions}{Total\ predictions}$

Per-Class Precision and Recall

$$\boldsymbol{Precision_i} = TP_i/(TP_i + FP_i) \qquad \boldsymbol{Recall_i} = TP_i/(TP_i + FN_i)$$

Macro-Averaging: Averages metrics equally across classes.

Micro-Averaging: Aggregates the contributions of all classes to compute the average metric, weighting them according to the number of instances

Per-Class Metrics

True Positives TP_i :

- Instances correctly predicted as class ii.

False Positives FP_i :

- Instances of other classes incorrectly predicted as class ii.

False Negatives FN_i :

- Instances of class i incorrectly predicted as other classes.

Per-Class Precision and Recall:

- Precision for Class i:

  $\boldsymbol{Precision_i} = TP_i/(TP_i + FP_i)$
- Recall for Class i:

  $\boldsymbol{Recall_i} = TP_i/(TP_i + FN_i)$

Macro-Averaging

- Equal weight to each class.

Macro Precision:

$$Precision_{macro} = \frac{1}{C}\sum_{i=1}^{C} Precision_i$$

Macro Recall:

$$Recall_{macro} = \frac{1}{C}\sum_{i=1}^{C} Recall_i$$

Micro-Averaging

- Global cumulative metrics.

Micro Precision:

$$Precision_{micro} = \sum_{i=1}^{C} TP_i \Big/ \sum_{i=1}^{C} (TP_i + FP_i)$$

Micro Recall:

$$Recall_{mIcro} = \sum_{i=1}^{C} TP_i \Big/ \sum_{i=1}^{C} (TP_i + FN_i)$$

When to Use Which Metric

Use **Macro-Averaging** when**:**

- Classes are equally important.
- Interested in performance across all classes.

Use **Micro-Averaging** when**:**

- Overall performance is the focus.
- Dealing with class imbalance.

Example Calculation

Assume a 3-Class Problem

Class	TP_i	FP_i	FN_i	$Precision_i$	$Recall_i$
A	40	10	5	40/(40+10)	40/(40+5)
B	30	15	10	30/(30+15)	30/(30+10)
C	20	5	15	20/(20+5)	20/(20+15)

Compute Macro and Micro Metrics

$$\boldsymbol{Precision_{macro}} = \frac{(Pr_A + Pr_B + Pr_C)}{3} = 0.76$$

$$\boldsymbol{Recall_{macro}} = \frac{(R_A + R_B + R_C)}{3} = 0.74$$

For micro-averaged metrics, sum up the TP, FP, and FN across all classes:

$$TP_{micro} = 40 + 30 + 20 = 90 \qquad FP_{micro} = 10 + 15 + 5 = 30 \qquad FN_{micro} = 5 + 10 + 15 = 30$$

$$\boldsymbol{Precision_{micro}} = \frac{TP_{micro}}{TP_{micro} + FP_{micro}} = \frac{90}{90 + 30} = 0.75$$

$$\boldsymbol{Recall_{micro}} = \frac{TP_{micro}}{TP_{micro} + FN_{micro}} = \frac{90}{90+30} = 0.75$$

Live Coding & Demos

Multilabel Classification

- ❌ Avocado
- ❌ Bird
- ✅ Cat
- ✅ Dog
- ❌ Engineer
- ❌ Fire
- ❌ Gnome
- ✅ Helicopter
- ❌ Jaguar
- ✅ Outdoor
- ✅ River
- ❌ Sausage
- ❌ House

Understanding Multilabel Classification

Definition:

- Each instance can belong to multiple classes simultaneously.

Examples:

- **Image Tagging**: An image may contain "cat," "dog," and "outdoor.“
- **Text Classification**: An article might belong to "science," "technology," and "health.“

Key Differences from Multiclass:

- In multilabel classification, **labels are not mutually exclusive**. Each instance can have multiple labels, unlike multiclass classification, where each instance belongs to only one class.

Evaluation Metrics for Multilabel Classification

Example-Based Metrics	**Label-Based Metrics**
evaluate each **instance** (example) **independently**, considering the entire set of predicted labels for that instance	evaluate each **label** (class) **independently**, aggregating the results across all labels.
useful when you want to assess the **model's performance on a per-instance basis.**	useful when you want to assess **performance for each individual label** in a multilabel setting

Example-Based Metrics

Metric	Description	Formula	Interpretation
Hamming Loss	Fraction of **misclassified** labels (both FP and FN)	$\frac{1}{NL}\sum_{i=1}^{N}\sum_{j=1}^{L} 1(\boldsymbol{y_{ij}} \neq \widehat{\boldsymbol{y_{ij}}})$	Lower values indicate fewer misclassifications.
Exact Match Ratio (EMR)	Proportion of instances with perfectly predicted label sets	$\frac{1}{N}\sum_{i=1}^{N} 1(\boldsymbol{y_i} = \widehat{\boldsymbol{y_i}})$	Strict metric; all labels must match exactly.
1/0 Loss	Proportion of instances with at least one misclassified label	$1 - \frac{1}{N}\sum_{i=1}^{N} 1(any\ \boldsymbol{y_i} \neq \widehat{\boldsymbol{y_i}})$	Sensitive to even a single misclassification per instance.
Precision	Proportion of predicted positive labels that are correct.	$\frac{1}{N}\sum_{i=1}^{N} \frac{\lvert \boldsymbol{y_i} \cap \widehat{\boldsymbol{y_i}} \rvert}{\lvert \widehat{\boldsymbol{y_i}} \rvert}$	how often the labels predicted as positive are correct, on average.
Recall	Measures the proportion of actual positive labels that are correctly predicted.	$\frac{1}{N}\sum_{i=1}^{N} \frac{\lvert \boldsymbol{y_i} \cap \widehat{\boldsymbol{y_i}} \rvert}{\lvert \boldsymbol{y_i} \rvert}$	how well the model identifies all relevant labels, on average.
F1 Score	Harmonic mean of precision and recall for each instance	$2\frac{Precison \times Recall}{Precison + Recall}$	balances precision and recall

*** Here, y_i represents the true set of labels for instance i and $\widehat{y_i}$ is the predicted set of labels.**
1 returns 1 if there is a mismatch (either a false positive or a false negative) and 0 if they match.
N – number of examples, L – number of labels.

Calculation Example

Instance	True Labels	Predicted Labels	Hamming Loss	Exact Match	1/0 Loss	Precision	Recall	F1-score
1	{cat, dog, tree}	{cat, dog}	0.33	0	1	1	0.67	0.8
2	{fish, tree}	{fish, tree}	0	1	0	1	1	1
3	{bird, fish}	{bird, tree}	0.5	0	1	0.5	0.5	0.5
		Average	**0.28**	**0.33**	**0.67**	**0.83**	**0.72**	**0.77**

Precision and Recall Example Calculation

Precision measures the proportion of predicted positive labels that are correct.

Calculation

1. **Instance 1**: True labels = {cat, dog, tree}, Predicted labels = {cat, dog}
    1. Correct labels (true positives): 2 (cat, dog)
    2. Total predicted labels: 2
    3. Precision for Instance 1: 2/2=1.0
2. **Instance 2**: True labels = {fish, tree}, Predicted labels = {fish, tree}
    1. Correct labels (true positives): 2 (fish, tree)
    2. Total predicted labels: 2
    3. Precision for Instance 2: 2/2=1.0
3. **Instance 3**: True labels = {bird, fish}, Predicted labels = {bird, tree}
    1. Correct labels (true positives): 1 (bird)
    2. Total predicted labels: 2
    3. Precision for Instance 3: 1/2=0.5

Recall measures the proportion of actual positive labels that are correctly predicted.

Calculation

1. **Instance 1**: True labels = {cat, dog, tree}, Predicted labels = {cat, dog}
    1. Correct labels (true positives): 2 (cat, dog)
    2. Total true labels: 3
    3. Recall for Instance 1: 2/3≈0.667
2. **Instance 2**: True labels = {fish, tree}, Predicted labels = {fish, tree}
    1. Correct labels (true positives): 2 (fish, tree)
    2. Total true labels: 2
    3. Recall for Instance 2: 2/2=1.0
3. **Instance 3**: True labels = {bird, fish}, Predicted labels = {bird, tree}
    1. Correct labels (true positives): 1 (bird)
    2. Total true labels: 2
    3. Recall for Instance 3: 1/2=0.5

Hamming Loss, 1/0 Loss, Exact Match Ratio Example Calculation

Hamming Loss is the fraction of misclassified labels, including both false positives and false negatives, across all labels and instances.

Calculation

1. **Instance 1**: True labels = {cat, dog, tree}, Predicted labels = {cat, dog}
    1. Misclassified labels: 1 (tree is missing)
    2. Hamming Loss for Instance 1: 1/3≈0.333
2. **Instance 2**: True labels = {fish, tree}, Predicted labels = {fish, tree}
    1. Misclassified labels: 0
    2. Hamming Loss for Instance 2: 0/2=0
3. **Instance 3**: True labels = {bird, fish}, Predicted labels = {bird, tree}
    1. Misclassified labels: 1 (tree is incorrect)
    2. Hamming Loss for Instance 3: 1/2=0.5

1/0 Loss measures the proportion of instances with at least one misclassified label.

Calculation

1. **Instance 1**: Misclassified label (tree missing).
2. **Instance 2**: No misclassification.
3. **Instance 3**: Misclassified label (tree is incorrect).

Exact Match Ratio (EMR) is the proportion of instances where the predicted set of labels matches the true set exactly.

Calculation

1. **Instance 1**: Not an exact match (missing tree).
2. **Instance 2**: Exact match.
3. **Instance 3**: Not an exact match (incorrectly predicted tree).

Label-Based Metrics

Metric	Description	Best Used When
Micro-Averaged Metrics	Metrics **weighted by label frequency**, emphasizing performance on more common labels. Suitable for evaluating overall effectiveness across all labels, especially in imbalanced class distributions.	Focus is on the overall performance across all labels, particularly in cases with imbalanced class distributions.
Macro-Averaged Metrics	Metrics that **treat each label equally**, providing an unweighted average. May overemphasize rare labels if they are poorly predicted.	Each label is equally important, regardless of frequency. Suitable when class importance is uniform across the dataset.
Label Ranking Average Precision (LRAP)	A specialized metric that evaluates the ranking of labels, measuring **how well relevant labels are ranked higher than irrelevant ones**. Useful for recommendation or tagging systems where the order of label relevance is important.	Used in applications like **recommendation or tagging systems** where label ranking order matters, such as prioritizing relevant tags.

Metrics Calculation

Metric	Formula	Interpretation
Micro-Averaged Precision	$\sum_{j=1}^{L} TP_j \,/\, \sum_{j=1}^{L} (TP_j + FP_j)$	Measures overall precision across labels, weighted by each label's frequency.
Micro-Averaged Recall	$\sum_{j=1}^{L} TP_j \,/\, \sum_{j=1}^{L} (TP_j + FN_j)$	Measures overall recall across labels, weighted by each label's frequency.
Micro-Averaged F1 Score	$2\frac{Precision_{micro} \times Recall_{micro}}{Precision_{micro} + Recall_{micro}}$	Balances precision and recall across labels.
Macro-Averaged Precision	$\frac{1}{L}\sum_{j=1}^{L} Precision_j$	Treats each label equally, useful when labels have equal importance.
Macro-Averaged Recall	$\frac{1}{L}\sum_{j=1}^{L} Recall_j$	Treats each label equally, useful when labels have equal importance.
Macro-Averaged F1 Score	$\frac{1}{L}\sum_{j=1}^{L} F1_j$	Provides balanced view across all labels, treating each label equally.
Label Ranking Average Precision (LRAP)	$\frac{1}{N}\sum_{i=1}^{N} \frac{1}{\lvert y_i \rvert}\sum_{j:y_{ij}=1}^{N} \frac{\lvert\{k: \widehat{y_{ik}} \geq \widehat{y_{ij}} \text{ and } y_{ik} = 1\}\rvert}{\lvert rank_{ij} \rvert}$	Measures how well the model ranks relevant labels higher than irrelevant ones for each instance.

Label Ranking Average Precision (LRAP)

Interpretation

- **High LRAP**: Indicates that the model consistently ranks relevant labels higher than irrelevant labels, meaning the model is effective at identifying and prioritizing the true labels for each instance.
- **Low LRAP**: Suggests that the model often ranks irrelevant labels higher than relevant ones, which could mean that the model struggles with prioritizing true labels accurately.

Advantages of LRAP

- **Ranking Sensitivity**: LRAP is sensitive to the ranking of labels, making it suitable for applications where the order of predicted labels matters.
- **Focus on Relevant Labels**: It measures performance based only on relevant (true) labels, so it doesn’t penalize the model for irrelevant labels as long as they are ranked lower than the relevant ones.

Label Ranking Average Precision (LRAP)

$$LRAP = \frac{1}{N}\sum_{i=1}^{N}\frac{1}{|y_i|}\sum_{j:y_{ij}=1}^{N}\frac{|\{k: \widehat{y_{ik}} \geq \widehat{y_{ij}}\ and\ y_{ik} = 1\}|}{|rank_{ij}|}$$

Where:

- N is the number of instances.
- y_i represents the true labels for instance i.
- $\widehat{y_i}$ represents the model's predicted scores or rankings for instance i.
- $y_{ij} = 1$ if label j is a true label for instance i.
- $rank_{ij}$ is the rank of the predicted score for label j in instance i.
- $|y_i|$ is the number of true labels for instance i.

The fraction inside the sum calculates the proportion of correctly ranked relevant labels for each instance. LRAP averages this value across all instances to give an overall score.

Multilabel Classification Calculation

Label	TP	FP	FN	Precision	Recall	F1 Score
cat	1	0	0	1	1	1
dog	1	0	0	1	1	1
tree	1	1	1	0.5	0.5	0.5
fish	1	0	1	1	0.5	0.67
bird	1	0	0	1	1	1
			Average	0.9	0.8	0.83

Multilabel Classification Models

Problem Transformation Methods:

- Binary Relevance (BR):
  - Treats each label as an independent binary classification problem.
  - **Pros:** Simple and versatile; works with any binary classifier.
  - **Cons:** Ignores correlations between labels.
- Classifier Chains (CC):
  - Each label's prediction depends on previous labels in a chain.
  - **Pros:** Captures label dependencies.
  - **Cons:** Sensitive to label order; computationally intensive with many labels.
- Label Powerset (LP):
  - Treats each unique combination of labels as a single class.
  - **Pros:** Naturally captures dependencies.
  - **Cons:** Poor scalability with many combinations; can overfit.

Multilabel Classification Models

Algorithm Adaptation Methods:

- Multilabel k-Nearest Neighbors (ML-kNN):
  - Adapts k-NN to multilabel by analyzing neighbors' labels.
  - **Pros:** Simple, interpretable; suitable for smaller datasets.
  - **Cons:** Memory-intensive; limited at capturing complex dependencies.
- Multilabel Decision Trees:
  - Extends decision trees to support multiple labels per instance.
  - **Pros:** Can model label dependencies; interpretable.
  - **Cons:** Requires more data; can overfit with small datasets.
- Multilabel Support Vector Machines (SVMs):
  - Treats each label as a binary classification task.
  - **Pros:** Effective for high-dimensional data.
  - **Cons:** Computationally intensive; requires tuning.

Multilabel Classification Models

Ensemble Methods:

- Random k-Labelsets (RAkEL):
  - Uses multiple Label Powerset classifiers on random subsets of labels.
  - **Pros:** Improves scalability of LP, captures dependencies.
  - **Cons:** Complexity increases with more labels.
- Ensemble of Classifier Chains (ECC):
  - Trains multiple Classifier Chains with random label orders.
  - **Pros:** Robust to label ordering, captures dependencies well.
  - **Cons:** High computational and memory cost.

Live Coding & Demos

Introduction to Ranking Problems and Metrics

Understanding Ranking Problems and Ordinal Classification

Ranking Problems: Scenarios where the order or ranking of predictions matters, rather than just predicting discrete classes.

- Examples: Search engine results, recommendation systems, and question answering.

Ordinal Classification: A subtype of ranking where classes have a natural order (e.g., "low," "medium," "high").

- **! Unlike regular classification, ordinal classification recognizes that certain classes are closer in meaning than others.**
- **Examples**: Movie ratings (1 star to 5 stars)
- Customer satisfaction (e.g., "very dissatisfied" to "very satisfied")
- Disease severity levels (e.g., "mild," "moderate," "severe")

Evaluation Metrics for Ordinal Classification

- **Mean Absolute Error (MAE):**
  - Measures the average absolute difference between predicted and true class values.
  - Penalizes larger misclassifications more heavily (e.g., predicting "low" for a "high" instance).
- **Mean Squared Error (MSE):**
  - Squares the differences, further penalizing larger errors.
- **Quadratic Weighted Kappa (QWK):**
  - Assesses agreement between predicted and actual classes, considering the ordinal structure.
  - Popular in applications where minimizing large discrepancies is critical.
- **Spearman's Rank Correlation:**
  - Evaluates the correlation between predicted and true ranks, ideal when relative ordering is more important than exact predictions.

Additionally: https://aclanthology.org/2020.acl-main.363.pdf

Techniques for Handling Ordinal Classification

- **Ordinal Logistic Regression:**
  - An extension of logistic regression designed for ordered classes.
  - Predicts probabilities across ordered class boundaries, rather than a single class.
  - **Pros:** Captures the ordinal nature of the data.
  - **Cons:** Requires more complex modeling.

- **Threshold-Based Regression:**
  - Train a regression model to predict a continuous score, then apply thresholds to map the score to ordered classes.
  - Allows customization of class boundaries based on data distribution.
  - **Pros:** Flexible with class boundaries, maintains order.
  - **Cons:** Requires careful threshold selection.

- **Pairwise Comparison or Ordinal Ensemble Methods:**
  - Compares pairs of classes to enforce order constraints (e.g., predicting if one class is higher or lower than another).
  - **Pros:** Captures complex dependencies in ordinal structure.
  - **Cons:** Computationally intensive, especially for many classes.

Live Coding & Demos

Key Takeaways

Class Imbalance in Binary Classification:

- Challenge: Imbalance often skews models toward the majority class, leading to poor performance on the minority class.
- Solutions: Use metrics like Precision, Recall, F1-Score, and PR AUC to focus on minority class performance. Techniques such as resampling, class weights adjustment, and cost-sensitive learning help create balanced models that capture the minority class better.

Overview of Multiclass and Multilabel Classification:

- Multiclass: Involves multiple mutually exclusive classes.
- Multilabel: Instances can have multiple labels, requiring metrics that respect label dependencies.

Overview of Ordinal Classification:

- Ordinal Classes: Ordered classes (e.g., "low" < "medium" < "high") require specialized techniques and metrics.
- Techniques: Ordinal Logistic Regression, Threshold-Based Regression, and Pairwise Comparison capture the natural order of classes.

Thank you for your attention

Appendix A

Contingency table

Contingency table

Sources: [1][2][3][4][5][6][7][8] view · talk · edit

		Predicted condition			
	Total population $= P + N$	**Predicted positive (PP)**	**Predicted negative (PN)**	Informedness, bookmaker informedness (BM) $= TPR + TNR - 1$	Prevalence threshold (PT) $= \frac{\sqrt{TPR \times FPR} - FPR}{TPR - FPR}$
Actual condition	**Positive (P)** [a]	**True positive** (TP), hit[b]	**False negative** (FN), miss, underestimation	True positive rate (TPR), recall, sensitivity (SEN), probability of detection, hit rate, power $= \frac{TP}{P} = 1 - FNR$	False negative rate (FNR), miss rate type II error [c] $= \frac{FN}{P} = 1 - TPR$
	Negative (N)[d]	**False positive** (FP), false alarm, overestimation	**True negative** (TN), correct rejection[e]	False positive rate (FPR), probability of false alarm, fall-out type I error [f] $= \frac{FP}{N} = 1 - TNR$	True negative rate (TNR), specificity (SPC), selectivity $= \frac{TN}{N} = 1 - FPR$
	Prevalence $= \frac{P}{P + N}$	Positive predictive value (PPV), precision $= \frac{TP}{PP} = 1 - FDR$	False omission rate (FOR) $= \frac{FN}{PN} = 1 - NPV$	Positive likelihood ratio (LR+) $= \frac{TPR}{FPR}$	Negative likelihood ratio (LR−) $= \frac{FNR}{TNR}$
	Accuracy (ACC) $= \frac{TP + TN}{P + N}$	False discovery rate (FDR) $= \frac{FP}{PP} = 1 - PPV$	Negative predictive value (NPV) $= \frac{TN}{PN} = 1 - FOR$	Markedness (MK), deltaP (Δp) $= PPV + NPV - 1$	Diagnostic odds ratio (DOR) $= \frac{LR+}{LR-}$
	Balanced accuracy (BA) $= \frac{TPR + TNR}{2}$	F_1 score $= \frac{2\,PPV \times TPR}{PPV + TPR} = \frac{2\,TP}{2\,TP + FP + FN}$	Fowlkes–Mallows index (FM) $= \sqrt{PPV \times TPR}$	Matthews correlation coefficient (MCC) $= \sqrt{TPR \times TNR \times PPV \times NPV} - \sqrt{FNR \times FPR \times FOR \times FDR}$	Threat score (TS), critical success index (CSI), Jaccard index $= \frac{TP}{TP + FN + FP}$

https://en.wikipedia.org/wiki/Evaluation_of_binary_classifiers